**BurgerFlow**
SABOR QUE FLUYE

DOCUMENTO TÉCNICO · SEGURIDAD DE APLICACIONES

# Modelo, CRUD y seguridad

Análisis de la entidad Pedido, sus permisos y la protección de los datos de entrega.

**Entidad principal** · Pedido
**Arquitectura analizada** · Express + SQLite
**Fecha de actualización** · 11 de noviembre de 2026
**Repositorio** · Viccct0rr/burgerflow-act-1-h4



INFORME ACADÉMICO · DOCUMENTACIÓN BASADA EN EL CÓDIGO

01 · CONTEXTO

# Resumen del sistema

BurgerFlow conecta el menú y la personalización de productos con el carrito/POS, la preparación de cocina y el reparto. La aplicación ejecutable es Node.js/Express con páginas HTML, CSS y JavaScript, y persiste las operaciones en SQLite; no es una aplicación React.

El prototipo ofrece operaciones de catálogo, inventario y pedidos con control de acceso por roles. No procesa pagos reales y no debe considerarse un servicio listo para producción. La identidad visual de la interfaz utiliza rojo, naranja y carbón; este informe mantiene esos colores para comunicar el mismo producto.

> **Entidad elegida: Pedido.** Conecta al cliente con los productos, el inventario, la cocina y la entrega. Conserva líneas e importes históricos; la información de dirección se separa para reducir su exposición.

## Alcance y evidencia

El análisis distingue entre el esquema implementado en SQLite, los controles efectivamente aplicados por Express y las recomendaciones para producción. Las relaciones conceptuales se comparan con el modelo real, sin asumir una base PostgreSQL ni una integración con Supabase.

| IMPLEMENTADO | SIMULADO / LOCAL | NO IMPLEMENTADO |
|---|---|---|
| SQLite, API Express, sesiones, roles, pedidos, inventario y autorización por servidor. | Cuentas y datos ficticios; borradores del carrito en sessionStorage hasta confirmar. | Pasarela real, mapas, recetas de ingredientes y publicación segura en producción. |

## Convenciones del modelo

- Importes monetarios: enteros en centavos de boliviano.
- Campos “obligatorios” según restricciones del esquema y validaciones de API.
- La dirección y el contacto son datos sensibles; se devuelven únicamente a roles autorizados según la relación con el pedido.

02 · ENTIDAD DE NEGOCIO

# Pedido y sus atributos

Los datos de entrega relacionados se detallan junto con la entidad. La obligatoriedad de `cliente_id` se aplica mediante trigger y validación de backend, aunque la migración aditiva preserve compatibilidad con registros anteriores.

### TABLA PRINCIPAL · ORDERS

| Campo | Tipo | Descripción | Obligatorio | Sensible |
|---|---|---|---|---|
| `orders.id` | Entero, PK | Identificador interno del pedido. | Sí | No |
| `orders.code` | Texto único | Código público del pedido. | Sí | No |
| `orders.created_by` | Entero, FK users | Cuenta autenticada que registró la operación; autor operativo. | Sí | Operativo |
| `orders.cliente_id` | Entero, FK users | Cliente titular; exige rol `customer`. Para un cliente autenticado, identidad tomada de la sesión. | Sí* | ID de cuenta |
| `orders.repartidor_id` | Entero, FK users, nullable | Cuenta activa asignada a la entrega; exige rol `delivery`. | No | Trabajador |
| `orders.order_type` | Texto enumerado | `dine_in`, `pickup` o `delivery`. | Sí | No |
| `orders.status` | Texto enumerado | Estado del flujo del pedido. | Sí | No |
| `orders.payment_method` | Texto enumerado | `cash`, `card` o `qr`; no almacena datos de tarjeta. | Sí | Transaccional |
| `subtotal_cents`, `tax_cents`, `service_cents`, `total_cents` | Enteros no negativos | Importes calculados por servidor; total sujeto a restricción de suma. | Sí | Financiero |
| `orders.created_at` | Fecha/hora textual | Fecha local de creación con valor por defecto. | Sí* | No |

*** Nota de esquema:** la migración es aditiva para conservar datos previos, por eso SQLite puede mostrar `cliente_id` como nullable en `table_info`; los triggers rechazan inserciones o actualizaciones nulas y validan el rol customer.

02 · DETALLE Y RELACIONES

# Líneas, personalización y entrega

| Campo | Tipo | Descripción | Obligatorio | Sensible |
|---|---|---|---|---|
| `order_items.id` | Entero, PK | Identificador de línea del pedido. | Sí | No |
| `order_id`, `product_id` | Enteros, FK | Pedido y producto relacionado. | Sí | No |
| `product_name` | Texto | Nombre del producto guardado como instantánea histórica. | Sí | No |
| `quantity` | Entero positivo | Unidades solicitadas y descontadas del stock del producto. | Sí | No |
| `unit_price_cents`, `line_total_cents` | Enteros | Precio oficial y total de línea congelados al confirmar. | Sí | Financiero |
| `personalizations` | Enteros, FK y texto | Opción, valor y diferencia permitida; el backend calcula el recargo. | Condicional | Preferencias |
| `delivery_info.id`, `order_id` | Entero, PK y FK única | Identificación de la fila de entrega y asociación uno a uno al pedido. | Condicional | Asociación sensible |
| `customer_name`, `phone`, `address` | Texto | Datos mínimos necesarios para entregar el pedido. | Solo delivery | **Sí** |
| `order_status_history` | Enteros, FK, texto, fecha | Autor, estado anterior/nuevo y momento del cambio para auditoría. | En cada cambio | Operativo |

### Relaciones del modelo implementado en SQLite

**USERS · customer**
Una cuenta cliente puede ser titular de varios pedidos.

1 → N
cliente_id

**ORDERS**
Cada pedido tiene un titular y cero o una asignación de delivery.

1 → N
order_id

**ORDER_ITEMS**
Un pedido requiere al menos una línea al crearse.

**USERS · delivery**
Cuenta activa con rol de repartidor.

0..1 → N
repartidor_id

**ORDERS + DELIVERY_INFO**
Datos de entrega separados para limitar exposición.

1 → 0..1
order_id único

**PERSONALIZATIONS**
Opciones dependientes de una línea.

**Modelo conceptual frente a implementación.** No hay tablas redundantes `clientes` o `repartidores`: las cuentas se reutilizan desde `users` y se distinguen por rol. `created_by` conserva quién registró el pedido y no sustituye al cliente titular. La dirección vive en `delivery_info`, no en la tabla de pedidos.

Las relaciones usan claves foráneas; triggers y backend verifican el rol asociado. Las líneas enlazan producto y pedido; las personalizaciones pueden revelar preferencias alimentarias. El inventario representa stock por producto terminado, no una receta de insumos.

03 · PERMISOS

# Matriz CRUD

Los roles implementados son administrador, cajero, cocina, delivery y cliente. La tabla resume quién puede operar cada dato; la autorización real se valida en Express, no por ocultar botones o rutas.

| Entidad o dato | Crear | Leer | Actualizar | Eliminar |
|---|---|---|---|---|
| **Catálogo** | Administrador | Público: activos; personal: catálogo | Administrador | Administrador: baja lógica; no se borra físicamente |
| **Inventario** | Administrador al crear producto | Administrador | Administrador: ajuste auditado; pedido: descuento transaccional | No permitido directamente |
| **Pedido y líneas** | Cliente, cajero o administrador | Admin/cajero: operación; cliente: propios; cocina: cola mínima; delivery: solo asignados | Cocina: preparación; delivery asignado: salida/entrega; admin según transiciones | No hay borrado físico; cancelación limitada a estados permitidos |
| **Personalizaciones** | Cliente/cajero/admin al crear | Vistas autorizadas; cocina recibe lo necesario | No disponible tras confirmar | Se conserva en historial |
| **Nombre, teléfono y dirección** | Cliente/cajero/admin en pedido delivery | Admin; cliente titular; repartidor asignado. No cocina ni cajero. | Cliente titular, cajero autor o admin, solo si está pending | Sin borrado individual; retención pendiente |
| **Usuarios** | Administrador vía API | Administrador vía API | Administrador: activar/desactivar; rol no editable | Baja lógica |
| **Roles y permisos** | Migración/datos iniciales | Sin endpoint específico | Sin interfaz/endpoint | Solo migración controlada |

## Estado de las operaciones

- **Persistente:** altas/ediciones/bajas lógicas de productos; consultas y ajustes de stock; pedidos, precios calculados por servidor, movimientos de inventario y estados de cocina/delivery.
- **Interfaz conectada:** menú, carrito, personalización, POS, inventario, cocina y reparto; el carrito temporal se mantiene en `sessionStorage` hasta confirmar.
- **API sin panel de administración:** creación/desactivación de usuarios y asignación manual de repartidor.
- **No implementado:** pasarela de pago, registro público de clientes, cambio de permisos, eliminación física, mapas de reparto y recetas de ingredientes.

> **Lectura de la matriz:** eliminar un pedido no equivale a cancelarlo. El historial se conserva; la cancelación permitida revierte inventario y registra el cambio.

04 · REGLA DE SEGURIDAD

# Proteger la dirección de entrega

> **Regla indispensable:** solo la persona propietaria, el personal con autorización administrativa o el repartidor asignado pueden consultar los datos de entrega. La identidad se toma de la sesión autenticada y la comprobación ocurre en el servidor.

## Por qué es necesaria

El nombre, teléfono y dirección permiten identificar y localizar a un cliente. Conocer o cambiar el ID del pedido no debe bastar para acceder a los datos. Ocultar una sección del HTML o un botón no constituye una autorización porque una solicitud puede enviarse directamente al endpoint.

## Aplicación comprobable en el código

**1 · Sesión**
Identifica la cuenta autenticada del repartidor

›

**2 · Pedido**
Consulta el registro por su ID

›

**3 · Asignación**
Compara repartidor_id con el ID de sesión

En `GET /api/orders/:id`, Express verifica el pedido y compara `orders.repartidor_id` con la identidad derivada de la sesión antes de serializar el contacto. La lista de delivery aplica el mismo filtro; cocina y cajero reciben proyecciones sin los datos de entrega. El backend no confía en un ID de usuario enviado por el navegador.

## Estado y límites

**Estado: implementada y verificada en el prototipo local.** El repartidor asignado puede consultar la entrega; el no asignado recibe un 404 genérico sin nombre, teléfono ni dirección. Esto no demuestra que un despliegue público esté protegido.

Para producción hacen falta HTTPS, secretos administrados fuera del repositorio, almacenamiento persistente con acceso restringido, sesiones seguras, respaldos, registros controlados y una política de retención/anonimización de datos personales.

> **Evidencia en código:** [app.js, sesión y autorización](../backend/app.js#L81); [GET de pedido individual](../backend/app.js#L317); [prueba de autorización por asignación](../backend/app.test.js#L169).

05 · PREGUNTA TÉCNICA

# ¿Puede un repartidor consultar un pedido ajeno?

## Pregunta

¿Puede un repartidor obtener la dirección de un pedido de otro repartidor cambiando el ID en `/api/orders/:id` o enviando otro ID de usuario desde el navegador?

## Respuesta basada en las pruebas

No en las rutas comprobadas: el servidor toma la identidad de la sesión, carga el pedido por ID y compara el repartidor asignado antes de devolver información. El usuario no asignado recibe `404 Pedido no encontrado`; no se serializan dirección, teléfono ni nombre. Añadir un identificador de usuario en la petición no sustituye la identidad autenticada.

## Prueba reproducible A/B

Ejecutar `npm run test:assignment`. El test usa cuentas ficticias, un entorno aislado y pedidos de prueba:

### Prueba A · Repartidor asignado

PERMITIDO · HTTP 200

Inicia sesión como repartidor A y consulta el pedido asignado. La respuesta incluye su información de entrega.

### Prueba B · Repartidor ajeno

RECHAZADO · HTTP 404

Repartidor B consulta el mismo pedido y variaciones de ID. No recibe ningún dato personal.

Las cuentas demo usan la contraseña local configurada por `DEMO_PASSWORD`; la prueba automatizada crea su propia base SQLite en memoria y credencial aislada. No utilizar cuentas ficticias como usuarios reales.

## Riesgo residual y recomendación

Los datos de demo y las pantallas no conectadas no son equivalentes a datos reales ni prueban controles de producción. GitHub Pages publica archivos estáticos pero no ejecuta el backend; no se debe publicar una base SQLite con datos personales. La asignación automática al primer repartidor activo solo sirve como regla demostrativa.

Mantener la autorización en cada consulta y modificación del servidor, devolver solo los campos indispensables, no confiar en precios del navegador y conservar pruebas de regresión cuando cambien rutas o permisos.

06 · CIERRE Y FUENTES

# Conclusiones

Pedido es el eje del proceso: identifica al cliente titular y al autor operativo, contiene una o varias líneas y puede tener cero o un repartidor asignado. Las direcciones se separan del pedido general y se sirven mediante autorización del backend.

El prototipo cuenta con SQLite persistente, operaciones CRUD selectivas, roles, sesiones, auditoría y flujos funcionales de inventario y pedidos. La prueba A/B verifica que el repartidor asignado recibe la dirección y el no asignado no. La suite actual tiene 12 pruebas automatizadas aprobadas.

> **Prioridades antes de usar datos reales:** desplegar API tras HTTPS, configurar secretos fuera del repositorio, proteger almacenamiento y respaldos, establecer retención/anonimización de datos y realizar una revisión de seguridad del despliegue.

## Fuentes de implementación

1. Repositorio BurgerFlow · código y documentación revisados.
2. backend/app.js, sesión y permisos.
3. backend/app.js, CRUD de producto e inventario.
4. backend/app.js, pedidos y cálculo transaccional.
5. backend/app.js, lectura individual y control de asignación.
6. Migración 001, esquema de pedidos y entrega; migración 003, cliente y repartidor.
7. backend/app.test.js, prueba de acceso asignado/no asignado.
8. backend/app.test.js, migración y claves foráneas.
9. mockup/api-client.js, carrito y creación de pedido.
10. README.md, ejecución local y arquitectura.